
Conditionally Trivial Special Member Functions

Contents

1. [Introduction and Revision History](#)
2. [Proposal](#)
 - 2.1. [Special member function candidates](#)
 - 2.2. [Standard Impact](#)
3. [Wording](#)
 - 3.1. [Default constructor](#)
 - 3.2. [Copy and move constructor](#)
 - 3.3. [Copy and move assignment](#)
 - 3.4. [Destructor](#)
4. [Acknowledgments](#)
5. [References](#)

§ 1. Introduction and Revision History

For a complete motivation for this proposal, see [P0848R0](#). In brief, it is important for certain class template instantiations to propagate the triviality from their template parameters - that is, we want `wrapper<T>` to be trivially copyable if and only if `T` is copyable. In C++17, this is possible, yet is extremely verbose. The introduction of Concepts provides a path to make this propagation substantially easier to write, but the current definition of trivially copyable doesn't quite suffice for what we want.

Consider:

```
template <typename T>
concept C = /* ... */;

template <typename T>
struct X {
    // #1
    X(X const&) requires C<T> = default;

    // #2
```

```
X(X const& ) { /* ... */ }  
};
```

According to the current working draft, both #1 and #2 are copy constructors. The current definition for trivially copyable requires that *each* copy constructor be either deleted or trivial. That is, we always consider both copy constructors, regardless of `T` and `C<T>`, and hence no instantiation of `X` is ever trivially copyable.

R0 of this paper, as presented in San Diego in November 2018, proposed to change the rules for trivially copyable such that we only consider the *best viable candidate* amongst the copy constructors given a synthesized overload resolution. That is, depending on `C<T>`, we either consider only #2 (because #1 wouldn't be viable) or only #1 (because it would be more constrained than #2 and hence a better match). However, EWG considered this to be confusing as trivially copyable is a property of a type and adding overload resolution simply adds more questions about context (e.g. do we consider accessibility?). EWG requested a new mechanism to solve this problem.

§ 2. Proposal

The following arguments and definitions are focused specifically on the copy constructor, but also apply similarly to the default constructor, the move constructor, the copy and move assignment operators, and the destructor.

§ 2.1. Special member function candidates

In the current working draft, the definition of copy constructor is, from [class.copy.ctor]:

A non-template constructor for class `X` is a copy constructor if its first parameter is of type `X&`, `const X&`, `volatile X&` or `const volatile X&`, and either there are no other parameters or else all other parameters have default arguments (9.2.3.6).

By this definition, both #1 and #2 in the previous example are copy constructors - regardless of `C<T>`. Instead, we could say that both of these functions are simply *candidates* to be copy constructors. For a given cv-qualification, a class can have multiple copy constructor candidates - but only have one copy constructor: the most constrained candidate of the candidates whose constraints are satisfied. If there is no most constrained candidate (that is, either there is no candidate or there is an ambiguity), then there is no copy constructor.

With this approach, #1 and #2 are both copy constructor candidates for the signature `X(X const&)`. For a given instantiation, if `C<T>` is not satisfied, there is only one candidate whose constraints are met and hence #2 is *the* copy constructor. If `C<T>` is satisfied, then #1 is the most constrained candidate and hence it is *the* copy constructor.

Using the example from R0:

```

template <typename T>
struct optional {
    // #1
    optional(optional const&)
        requires TriviallyCopyConstructible<T> && CopyConstructible<T>
        = default;

    // #2
    optional(optional const& rhs)
        requires CopyConstructible<T>
        : engaged(rhs.engaged)
    {
        if (engaged) {
            new (value) T(rhs.value);
        }
    }
};

```

We have two copy constructor candidates: **#1** and **#2**. For `T=unique_ptr<int>`, neither candidate has its constraints satisfied, so there is no copy constructor. For `T=std::string`, only **#2** has its constraints satisfied, so it is the copy constructor. For `T=int`, both candidates have their constraints satisfied and **#1** is more constrained than **#2**, so **#1** is the copy constructor.

With the introduction of the notion of copy constructor candidates, and reducing the meaning of "copy constructor" to be the most constrained candidate, no change is necessary to the definition of trivially copyable; requiring that each copy constructor be trivial or deleted becomes the correct definition - and meets the requirements of making it easy to propagate triviality.

§ 2.2. Standard Impact

One important question is: now that we have two terms, copy constructor candidate and copy constructor, what do we have to change throughout the standard? In the latest working draft, [N4778](#), there are 79 uses of the term *copy constructor*. 27 of those are in the core language (several of which are in notes or example), and 52 in the library.

Whenever the library uses "copy constructor," it really means this new refined definition of *the* copy constructor. Indeed, it's even more specific than that as the library only cares about the most constrained copy constructor candidate which takes an lvalue reference to `const`. Hence, no library wording needs to change all.

Of the 27 core language uses:

- 2 are seemingly unnecessary: `[class.temporary]/3` and `[expr.call]/12` contain a complete redefinition of what it means for a type to be trivially copyable.
- 2 refer to the new, proposed notion of copy constructor as the most constrained candidate

- 12 refer to what this paper proposes to call a copy constructor candidate, mostly in [class.copy.ctor] but also when describing the copyability of lambdas.
- 11 are non-normative, appearing in comments in examples or notes

In other words, introducing this notion of a copy constructor candidate and the copy constructor would primarily require simply changing [class.copy.ctor], which in the working draft describes the rules for what a copy constructor is and when it would be generated. Just about everything outside of that section would not only not need to change, but arguably be silently improved - we typically only care about the most constrained copy constructor candidate and now the wording would actually say that.

§ 3. Wording

Relative to N4791. Due to potential confusion with overload resolution, we are using the term "prospective" rather than the term "candidate" throughout.

Change 9.4.2 [dcl.fct.def.default], paragraph 1:

A function definition whose *function-body* is of the form `= default ;` is called an *explicitly-defaulted definition*. A function that is explicitly defaulted shall

- be a prospective special member function or a comparison operator ([expr.spaceship], [expr.rel], [expr.eq]), and
- not have default arguments.

Insert into 9.4.2 [dcl.fct.def.default], paragraph 5:

Explicitly-defaulted functions and implicitly-declared functions are collectively called *defaulted* functions, and the implementation shall provide implicit definitions for them ([class.ctor] [class.dtor], [class.copy.ctor], [class.copy.assign]), which might mean defining them as deleted. A defaulted prospective special member function that is not a special member function shall be defined as deleted. A function is *user-provided* if it is user-declared and not explicitly defaulted or deleted on its first declaration.

§ 3.1. Default constructor

Change 10.3.4.1 [class.default.ctor], paragraph 1:

A prospective default constructor for a class `X` is a constructor of class `X` for which each parameter that is not a function parameter pack has a default argument (including the case of a constructor with no parameters). If there is no user-declared constructor for class `X`, a non-explicit inline public constructor ~~having no parameters~~ is implicitly declared as defaulted (9.4). ~~An implicitly-declared default constructor is an inline public member of its class.~~ Such a constructor will have the form:

```
X()
```

A prospective default constructor is a default constructor if

- all of its constraints (if any) are satisfied, and
- it is at least as constrained as ([temp.constr.order]) every other prospective default constructor whose constraints (if any) are satisfied.

§ 3.2. Copy and move constructor

Introduce the concept of prospective copy constructor in 10.3.4.2 [class.copy.ctor], paragraph 1:

A non-template constructor for class `X` is a *prospective copy constructor* if its first parameter is of type `X&`, `const X&`, `volatile X&` or `const volatile X&`, and either there are no other parameters or else all other parameters have default arguments (9.2.3.6).

A prospective copy constructor is a copy constructor if

- all of its constraints (if any) are satisfied, and
- it is at least as constrained as ([temp.constr.order]) every other prospective copy constructor that has the same first parameter type and whose constraints (if any) are satisfied.

[Note: A class can have multiple copy constructors, provided they have different signatures -end note]

Introduce the concept of prospective move constructor in 10.3.4.2 [class.copy.ctor], paragraph 2:

A non-template constructor for class `X` is a *prospective move constructor* if its first parameter is of type `X&&`, `const X&&`, `volatile X&&`, or `const volatile X&&`, and either there are no other parameters or else all other parameters have default arguments (9.2.3.6).

A prospective move constructor is a move constructor if

- all of its constraints (if any) are satisfied, and
- it is at least as constrained as ([temp.constr.order]) every other prospective move constructor that has the same first parameter type and whose constraints (if any) are satisfied.

[Note: A class can have multiple move constructors, provided they have different signatures -end note]

Reduce the cases where we implicitly generate special members in 10.3.4.2 [class.copy.ctor], paragraph 6:

If the class definition does not explicitly declare a *prospective* copy constructor, a non-explicit one is declared implicitly. If the class definition declares a *prospective* move constructor or *prospective* move assignment operator, the implicitly declared *prospective* copy constructor is defined as deleted; otherwise, it is defined as defaulted (9.4). The latter case is deprecated if the class has a user-declared *prospective* copy assignment operator or a user-declared *prospective* destructor (D.5). [

Note: An implicitly-declared prospective copy constructor is a copy constructor. -end note]

Change 10.3.4.2 [class.copy.ctor], paragraph 7:

The implicitly-declared [prospective](#) copy constructor for a class `X` will have the form

```
X::X(const X&)
```

if each potentially constructed subobject of a class type `M` (or array thereof) has a copy constructor whose first parameter is of type `const M&` or `const volatile M&`. Otherwise, the implicitly-declared [prospective](#) copy constructor will have the form

```
X::X(X&)
```

Change 10.3.4.2 [class.copy.ctor], paragraph 8:

If the definition of a class `X` does not explicitly declare a [prospective](#) move constructor, a non-explicit one will be implicitly declared as defaulted if and only if

- `X` does not have a user-declared [prospective](#) copy constructor,
- `X` does not have a user-declared [prospective](#) copy assignment operator,
- `X` does not have a user-declared [prospective](#) move assignment operator, and
- `X` does not have a user-declared [prospective](#) destructor. [Note: [An implicitly-declared prospective move constructor is a move constructor](#). When ~~the~~ [a prospective](#) move constructor is not implicitly declared or explicitly supplied, expressions that otherwise would have invoked the move constructor may instead invoke a copy constructor. —end note]

Change 10.3.4.2 [class.copy.ctor], paragraph 9:

The implicitly-declared [prospective](#) move constructor for class `X` will have the form

```
X::X(X&&)
```

Change 10.3.4.2 [class.copy.ctor], paragraph 13:

Before ~~the~~ [a](#) defaulted copy/move constructor for a class is implicitly defined, all non-user-provided copy/move constructors for its potentially constructed subobjects shall have been implicitly defined.

Change 10.3.4.2 [class.copy.ctor], paragraph 14:

~~The~~ [An](#) implicitly-defined copy/move constructor for a non-union class `X` performs a memberwise copy/move of its bases and members. [Note: Default member initializers of non-static data members are ignored. See also the example in 10.9.2. —end note] The order of initialization is the same as the order of initialization of bases and members in a user-defined constructor (see 10.9.2). Let `x` be either the parameter of ~~the~~ [a copy](#) constructor or, for ~~the~~ [a](#) move constructor, an xvalue referring to the parameter. Each base or non-static data member is copied/moved in the manner appropriate to its type:

- if the member is an array, each element is direct-initialized with the corresponding subobject of `x`;
- if a member `m` has rvalue reference type `T&&`, it is direct-initialized with `static_cast<T&&>(x.m)`;
- otherwise, the base or member is direct-initialized with the corresponding base or member of `x`.

Virtual base class subobjects shall be initialized only once by **the** `an` implicitly-defined copy/move constructor (see 10.9.2).

Change 10.3.4.2 [class.copy.ctor], paragraph 15:

The `An` implicitly-defined copy/move constructor for a union `X` copies the object representation (6.7) of `X`.

§ 3.3. Copy and move assignment

Change 10.3.5 [class.copy.assign], paragraph 1:

A user-declared **prospective** copy assignment operator `X::operator=` is a non-static non-template member function of class `X` with exactly one parameter of type `X`, `X&`, `const X&`, `volatile X&`, or `const volatile X&`.

A prospective copy assignment operator is a copy assignment operator if

- all of its constraints (if any) are satisfied, and
- it is at least as constrained as ([temp.constr.order]) every other prospective copy assignment operator that has the same parameter type and whose constraints (if any) are satisfied.

[Note: An overloaded assignment operator must be declared to have only one parameter; see 11.5.3. —end note] [Note: More than one form of copy assignment operator may be declared for a class. —end note]

[Note: If a class `X` only has a copy assignment operator with a parameter of type `X&`, an expression of type `const X` cannot be assigned to an object of type `X`. [Example:

Change 10.3.5 [class.copy.assign], paragraph 2:

If the class definition does not explicitly declare a **prospective** copy assignment operator, one is declared implicitly. If the class definition declares a **prospective** move constructor or **prospective** move assignment operator, the implicitly declared **prospective** copy assignment operator is defined as deleted; otherwise, it is defined as defaulted (9.4). The latter case is deprecated if the class has a user-declared **prospective** copy constructor or a user-declared **prospective** destructor (D.5). The implicitly-declared **prospective** copy assignment operator for a class `X` will have the form

```
X& X::operator=(const X&)
```

if

- each direct base class `B` of `X` has a copy assignment operator whose parameter is of type `const B&`, `const volatile B&`, or `B`, and
- for all the non-static data members of `X` that are of a class type `M` (or array thereof), each such class type has a copy assignment operator whose parameter is of type `const M&`, `const volatile M&`, or `M`.

Otherwise, the implicitly-declared [prospective](#) copy assignment operator will have the form

```
X& X::operator=(X&)
```

Change 10.3.5 [class.copy.assign], paragraph 3:

A user-declared [prospective](#) move assignment operator `X::operator=` is a non-static non-template member function of class `X` with exactly one parameter of type `X&&`, `const X&&`, `volatile X&&`, or `const volatile X&&`.

[A prospective move assignment operator is a move assignment operator if](#)

- [all of its constraints \(if any\) are satisfied, and](#)
- [it is at least as constrained as \(\[temp.constr.order\]\) every other prospective move assignment operator that has the same parameter type and whose constraints \(if any\) are satisfied.](#)

[Note: An overloaded assignment operator must be declared to have only one parameter; see 11.5.3. —end note] [Note: More than one form of move assignment operator may be declared for a class. —end note]

Change 10.3.5 [class.copy.assign], paragraph 4:

If the definition of a class `X` does not explicitly declare a [prospective](#) move assignment operator, one will be implicitly declared as defaulted if and only if

- `X` does not have a user-declared [prospective](#) copy constructor,
- `X` does not have a user-declared [prospective](#) move constructor,
- `X` does not have a user-declared [prospective](#) copy assignment operator, and
- `X` does not have a user-declared [prospective](#) destructor.

[Example: The class definition

```
struct S {
    int a;
    S& operator=(const S&) = default;
};
```

will not have a default move assignment operator implicitly declared because ~~the~~ [a prospective](#) copy assignment operator has been user-declared. ~~The~~ [A prospective](#) move assignment operator may be explicitly defaulted.


```
struct S {
    int a;
    S& operator=(const S&) = default;
    S& operator=(S&&) = default;
};
```

—end example]

Change 10.3.5 [class.copy.assign], paragraph 5:

The implicitly-declared [prospective](#) move assignment operator for a class X will have the form

```
X& X::operator=(X&&);
```

Change 10.3.5 [class.copy.assign], paragraph 6:

~~The~~ [An](#) implicitly-declared [prospective](#) copy/move assignment operator for class X has the return type `X&`; it returns the object for which the assignment operator is invoked, that is, the object assigned to. An implicitly-declared [prospective](#) copy/move assignment operator is an inline public member of its class.

Change 10.3.5 [class.copy.assign], paragraph 8:

Because a [prospective](#) copy/move assignment operator is implicitly declared for a class if not declared by the user, a base class copy/move assignment operator is always hidden by the corresponding [prospective](#) assignment operator of a derived class ([over.ass]). A using-declaration ([namespace.udecl]) that brings in from a base class an assignment operator with a parameter type that could be that of a [prospective](#) copy/move assignment operator for the derived class is not considered an explicit declaration of such a [prospective](#) operator and does not suppress the implicit declaration of the derived class [prospective](#) operator; the operator introduced by the using-declaration is hidden by the implicitly-declared [prospective](#) operator in the derived class.

Change 10.3.5 [class.copy.assign], paragraph 10:

A copy/move assignment operator for a class X that is defaulted and not defined as deleted is implicitly defined when it is odr-used ([basic.def.odr]) (e.g., when it is selected by overload resolution to assign to an object of its class type), when it is needed for constant evaluation ([expr.const]), or when it is explicitly defaulted after its first declaration. ~~The~~ [An](#) implicitly-defined [prospective](#) copy/move assignment operator is constexpr if

- X is a literal type, and
- the assignment operator selected to copy/move each direct base class subobject is a constexpr function, and
- for each non-static data member of X that is of class type (or array thereof), the assignment operator selected to copy/move that member is a constexpr function.

Change 10.3.5 [class.copy.assign], paragraph 11:

Before ~~the~~ [a](#) defaulted copy/move assignment operator for a class is implicitly defined, all non-user-provided copy/move assignment operators for its direct base classes and its non-static data members shall have been implicitly defined. [*Note: An implicitly-declared copy/move assignment operator has an implied exception specification ([except.spec]).* —end note*]

Change 10.3.5 [class.copy.assign], paragraph 12:

~~The~~ [An](#) implicitly-defined copy/move assignment operator for a non-union class X performs memberwise copy/move assignment of its subobjects. The direct base classes of X are assigned first, in the order of their declaration in the base-specifier-list, and then the immediate non-static data members of X are assigned, in the order in which they were declared in the class definition. Let x be either the parameter of the function or, for the move operator, an xvalue referring to the parameter. Each subobject is assigned in the manner appropriate to its type:

- [...]

It is unspecified whether subobjects representing virtual base classes are assigned more than once by ~~the~~ [an](#) implicitly-defined copy/move assignment operator.

Change 10.3.5 [class.copy.assign], paragraph 13:

~~The~~ [An](#) implicitly-defined copy assignment operator for a union X copies the object representation ([basic.types]) of X.

§ 3.4. Destructor

Change 10.3.6 [class.dtor], paragraph 1:

In a declaration of a [prospective](#) destructor, the declarator is a function declarator (9.2.3.5) of the form [...] A [prospective](#) destructor shall take no arguments (9.2.3.5). Each *decl-specifier* of the *decl-specifier-seq* of a [prospective](#) destructor declaration (if any) shall be `friend`, `inline`, or `virtual`.

A prospective destructor is a *destructor* if

- all of its constraints (if any) are satisfied, and
- it is at least as constrained as ([temp.constr.order]) every other prospective destructor whose constraints (if any) are satisfied.

A class shall have a destructor.

Change 10.3.6 [class.dtor], paragraph 4:

If a class has no user-declared [prospective](#) destructor, a [prospective](#) destructor is implicitly declared as defaulted (9.4). An implicitly-declared [prospective](#) destructor is an inline public member of its class.

An implicitly-declared prospective destructor for a class X will have the form

```
~X()
```

Change 10.3.6 [class.dtor], paragraph 10:

A prospective destructor can be declared `virtual` (10.6.2) or pure `virtual` (10.6.3); if the destructor of a class is `virtual` and any objects of that class or any derived class are created in the program, the destructor shall be defined. If a class has a base class with a virtual destructor, its destructor (whether user- or implicitly-declared) is virtual.

§ 4. Acknowledgments

Thanks to Gaby dos Reis, Daveed Vandevoorde, and Jonathan Wakely for helping bring us to this design.
Thanks to Jens Maurer for the wording wizardry.

§ 5. References

- [\[N4778\]](#) "Working Draft, Standard for Programming Language C++" by Richard Smith, 2018-10-08
- [\[P0848\]](#) "Conditionally Trivial Special Member Functions" by Barry Revzin, Casey Carter, 2017-11-12